

Question: Compare Pickle and Joblib as serialisation choices for this deployment context. Which would you use and why? What risks does each carry when the model is loaded back in an environment that differs from the one it was trained in? \

Pickle

- Python's built-in serialization library.
- Can serialize almost any Python object.
- Good for small objects.
- Slower when saving/loading large NumPy arrays.

Joblib

- Built specifically for machine learning objects.
- Optimized for NumPy arrays.
- Faster loading and saving for large scikit-learn models.
- Supports memory mapping, which reduces RAM usage.

Which would I use?

For a **Flask service handling hundreds of requests per hour**, I would choose **Joblib** because:

- Most scikit-learn models contain large NumPy arrays.
- Faster model loading.
- Better performance in production.
- Widely recommended for scikit-learn deployment.

Risks when loading in another environment

Both Pickle and Joblib have similar risks.

1. **Version mismatch**
 - Different versions of scikit-learn may fail to load the model or produce incorrect predictions.
2. **Python version mismatch**
 - Different Python versions can break compatibility.
3. **Missing dependencies**
 - Required libraries may not exist in the deployment environment.
4. **Security risk**
 - Never load Pickle or Joblib files from untrusted sources because they can execute arbitrary Python code.

Question: A colleague suggests using a Variational Autoencoder (VAE), while another recommends a Stable Diffusion-based model. Compare the two approaches for this image-generation task — address their key architectural differences, output quality, and which is more appropriate for a production marketing pipeline.

Variational Autoencoder (VAE)

Architecture:

Encoder → Latent Space → Decoder

How it works:

- Compresses images into a latent representation.
- Decoder reconstructs the image.
- Learns probability distributions.

Advantages

- Fast training
- Stable training
- Useful for representation learning
- Generates smooth latent space

Disadvantages

- Images tend to be blurry.
- Lower realism.

Stable Diffusion

Architecture

Text Prompt

↓

Text Encoder

↓

Diffusion Process

↓

Denoising UNet

↓

VAE Decoder



High-quality Image

How it works

Starts from random noise.

Gradually removes noise until a realistic image is generated.

Advantages

- Extremely realistic images
- Sharp details
- Excellent text-image alignment
- State-of-the-art quality

Disadvantages

- Computationally expensive
- Slower generation

Question: GPT-style decoder-only models and BERT-style encoder-only models are both available to your team. Which architecture is better suited to each job, and what fundamental property of each architecture makes it the right choice for that specific task?

"What cuisines are trending near me?"

Needs

- Generate natural language
- Open-ended reasoning

Best model

GPT (Decoder-only)

Reason

GPT predicts the next token and is designed for text generation.

Task 2

Review sentiment classification

Positive

Neutral

Negative

Best model

BERT (Encoder-only)

Reason

BERT reads the entire sentence simultaneously using bidirectional attention.

It learns contextual meaning very well.

Classification accuracy is generally higher.

Fundamental Difference

GPT

- Decoder-only
- Left-to-right attention
- Excellent text generation
- Autoregressive

Used for

- Chatbots
- Content generation
- Question answering

BERT

- Encoder-only
- Bidirectional attention
- Understands complete sentence context
- Excellent for classification

Used for

- Sentiment analysis
- Spam detection
- NER
- Text classification

Question: Explain the difference between zero-shot and few-shot prompting. Given the constraints described, which strategy would you apply, and how would you structure the prompt to maximise the quality of the bot's responses?

Zero-shot Prompting

No examples are provided.

Example

Classify this complaint and respond politely.

Complaint:

"My order never arrived."

The LLM relies only on pre-trained knowledge.

Few-shot Prompting

Provide a few examples before asking the new question.

Example

Complaint:

Food was cold

Category:

Delivery Issue

Response:

We're sorry...

Complaint:

Missing drink

Category:

Missing Item

Response:

We'll refund...

Complaint:

Packaging damaged

??

Which should be used?

The company already has **3 annotated complaint-response pairs**.

Therefore,

Few-shot prompting is the best choice.

It gives the model examples of:

- category
- tone
- response style

This greatly improves consistency.

Prompt Structure

You are a customer support assistant.

Example 1

Complaint:

Wrong item delivered

Category:

Order Error

Response:

We're sorry...

Example 2

Complaint:

Missing cutlery

Category:

Packaging Issue

Response:

We apologize...

Example 3

Complaint:

Packaging damaged

Category:

Packaging Issue

Response:

We understand your frustration...

Now answer:

Complaint:

Customer complaint here.

Question: Describe what distinguishes a LangChain agent from a simple LLM chain. Given that your food ordering assistant must check live restaurant availability, call an API to calculate estimated delivery time, and validate discount codes, justify which approach is more appropriate — and identify two risks the agent approach introduces

LLMChain

Flow

User

↓

Prompt

↓

LLM

↓

Response

One single LLM call.

No external tools.

Static.

LangChain Agent

Flow

User



LLM



Reasoning



Chooses Tool



Calls API



Observes Result



Calls Another Tool



Final Answer

The LLM decides which tools to use.

Which is suitable?

Requirements

- Restaurant availability
- Delivery time API
- Discount validation

These require **real-time external information**.

Therefore,

LangChain Agent is the better choice.

Because it can:

- Call restaurant API
- Call delivery API
- Validate coupon API
- Combine results
- Produce one final response

An LLMChain cannot perform these dynamic actions by itself.

Risks of Agent

1. Higher latency

Multiple API calls increase response time.

2. More complexity

- Tool failures
- API errors
- Timeout handling
- More debugging

Question: Compare fine-tuning and RAG for this specific scenario. Address update frequency, cost, hallucination risk, and data privacy. Which approach is more suitable for a daily-updated, document-grounded Q&A bot, and what limitations does your recommended approach still carry?

Fine-Tuning

What it does

Updates the model weights using new training data.

Advantages

- Learns domain-specific style.
- Better task specialization.
- Faster inference after training.

Disadvantages

- Expensive training.
 - Needs retraining whenever data changes.
 - Difficult with daily updates.
 - Risk of outdated knowledge.
-

RAG (Retrieval-Augmented Generation)

How it works

User Question

↓

Embedding

↓

Vector Database

↓

Retrieve relevant documents

↓

LLM

↓

Grounded Answer

Advantages

- No retraining required.
- Daily document updates are easy.
- Lower hallucination.
- Answers are grounded in retrieved documents.
- Better privacy because documents stay in your own database.

Disadvantages

- Depends on retrieval quality.
- Slower than pure LLM.
- Requires maintaining a vector database.
- If retrieval misses relevant chunks, the answer may still be incomplete.

